Software Requirements and Software Architecture

이찬근

소프트웨어학부
중앙대학교

Functional and Non-Functional Requirements

- Larman's book: FURPS+

- Functional requirements (FR)
  - Describe **functionality** or **system services**
  - How the system should react to particular inputs and how the system should behave in particular situations

- Non-functional requirements (NFR)
  - Describe **properties** of the system or the domain
  - NFR further divides into
    - Quality Requirements (quality attributes): Usability, Reliability, Performance, etc.
    - Constraints: OS, hardware, development languages, etc. (+)
  - Non-functional requirements may be more critical than functional requirements. **If these are not met, the system is useless**.

Non-Functional Requirements

- May affect the overall architecture of a system
  - e.g. **performance requirements** → organize the system to minimize communications between components

시스템 아키텍처에 영향.

- May generate a number of related functional requirements
  - e.g. **security** requirement

↳ FR과 관련된 것을 생성

- May generate requirements restricting existing requirements

↳ 기존 requirement를 제한

Quality Requirements and Architecture

- Functionality does not determine architecture.
  - Given a set of required functionality, there is no end to the architectures you could create to satisfy that functionality.
  - E.g., you could divide up the functionality in any number of ways and assign the sub-pieces to different architectural elements.
  - In fact, if functionality were the only thing that mattered, you wouldn't have to divide the system into pieces at all.

- Instead, we design our systems
  - as structured sets of cooperating architectural elements (layers, components, classes, databases, apps, threads, peers, tiers, and on and on)
  - to support NFRs (Quality attributes and Constraints)

Software architecture in practice, 3rd edition(2012)

Quality Requirement Description

- A system will be “modifiable” ➔ too ambiguous
  - because every system is modifiable with respect to some changes and not modifiable with respect to others.

❖ Quality attributes

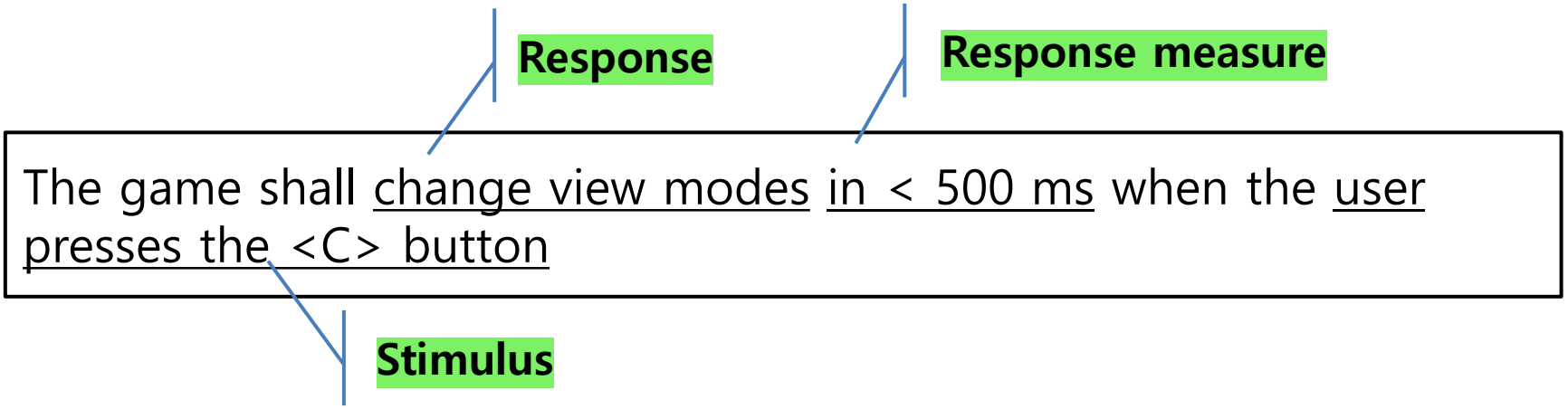
- Measurable or testable properties of a system
- that are used to indicate how well the system satisfies the needs of its stakeholders.

- Solution: **Quality Attribute Scenarios (QAS)**

Basic Quality Attribute Scenario

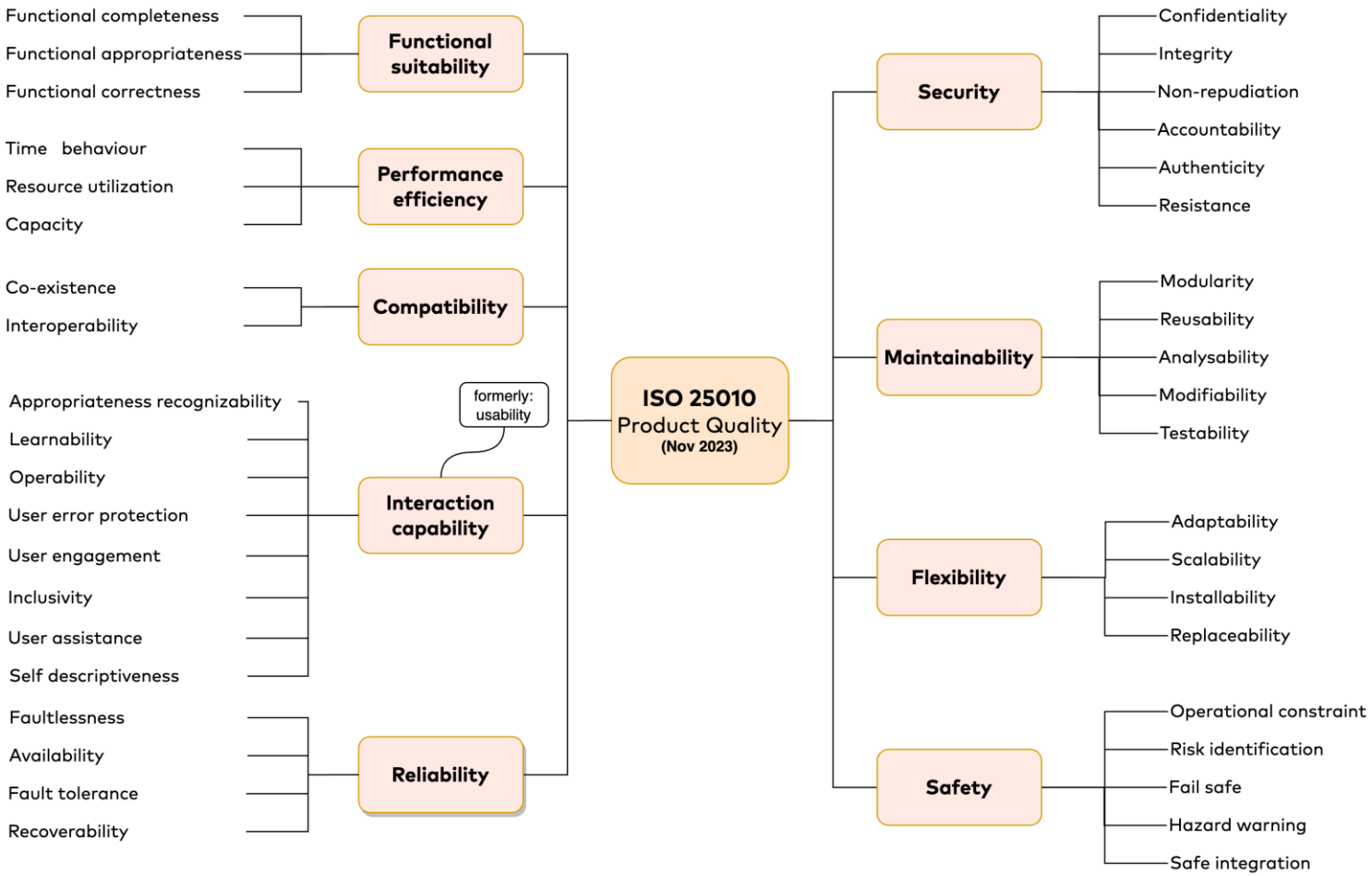
- A short description of how(**measure**) a system is required to respond(**response**) to some event(**stimulus**).

Example of Performance Scenario:



Designing software architecture-A practical approach(2016)

ISO/IEC 25010:2023 Quality Model



ISO/IEC 25010:2013 Systems and software Quality Requirements and Evaluation (SQuaRE) - System and Software Quality Model

Availability

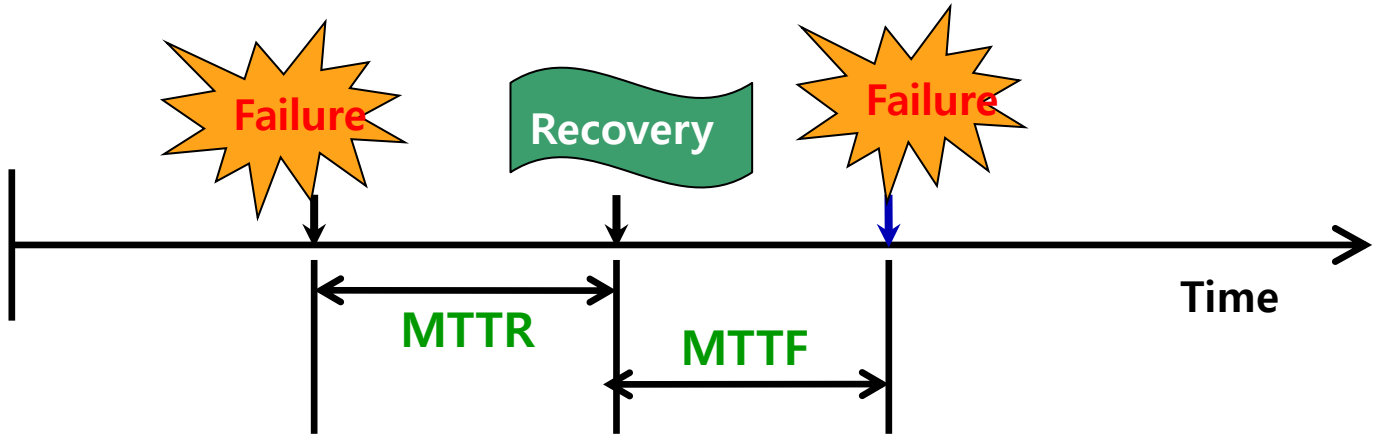
The degree to which a system or component is **operational and accessible when required for use**

[IEEE 610]

$$\text{Availability} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$$

MTBF : Mean Time Between **Failures**

MTTR : Mean Time To **Repair**



Quality Attribute Scenario (QAS) 시나리오

- The best way to discuss, document, and prioritize quality requirement is as a set of scenarios → Quality requirement 를 시나리오들로 표현.

- Scenario describes the system's response to some stimulus.

- Example of modifiability
  - One can specify the modifiability response measure you would like to achieve (say, elapsed time or effort) in response to a specific change request.
  - "a change to update shipping rates on the e-commerce website is completed and tested in less than 1 person-day of effort"

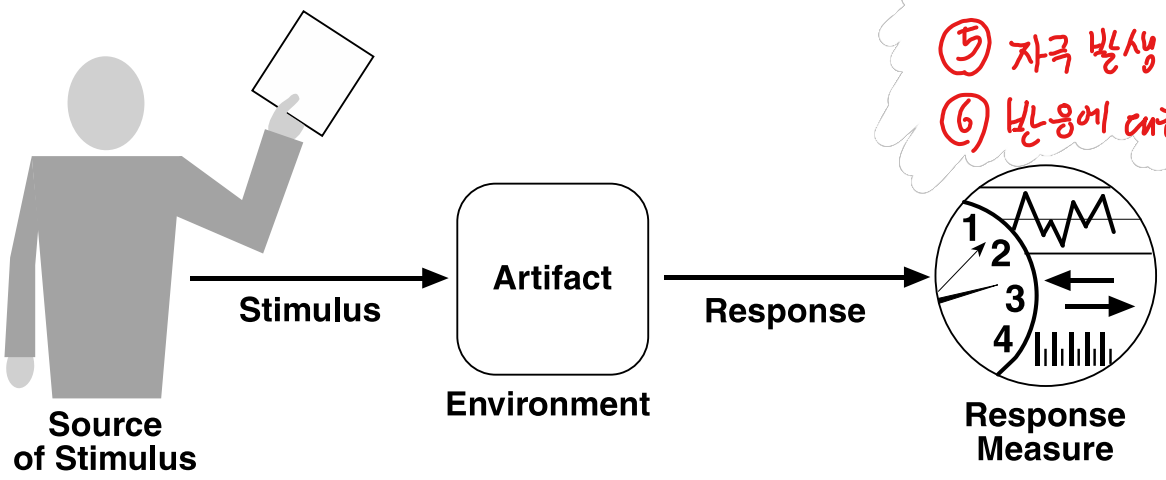
Designing software architecture-A practical approach(2016)

Complete Quality Attribute Scenario

(Complete QAS) → QAS 3요소 + (3개 추가 QAS)

- In addition to **stimulus**, **response** and **response measure**, complete QA scenario adds three other parts:
  - The **source** of the stimulus: the user
  - The **artifact** affected: the entire system
  - The **environment**: in normal operation, startup, degraded mode, or some other mode?
- In total, there are six parts of a complete QA scenario.

① 자극 → ② 누가 자극을 줬나?
③ 반응 → ④ 누가 반응했나?
⑤ 자극 발생 조건
⑥ 반응에 대한 측정 값.



Software architecture in practice, 3rd edition(2012)

Complete Quality Attribute Scenario

- Six-part quality attribute scenario framework for recording, negotiating, and analyzing quality attribute requirements.

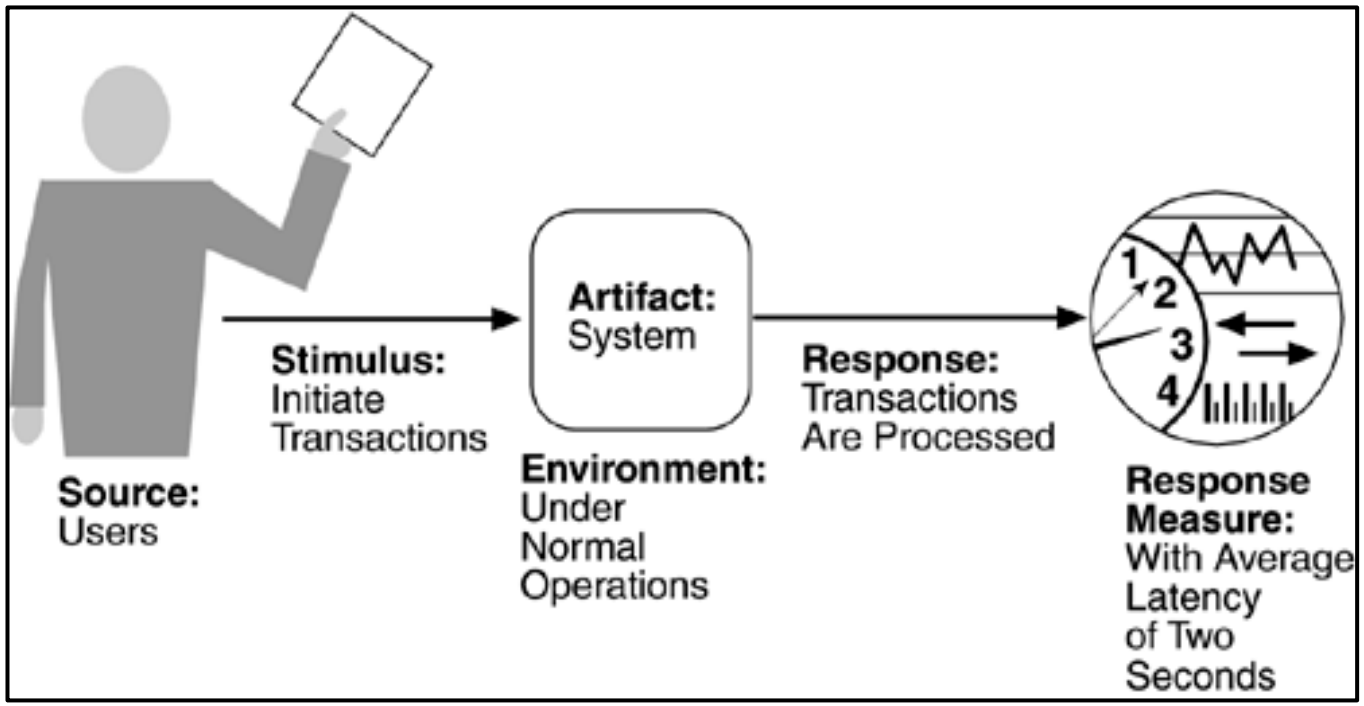
Stimulus	The stimulus is a condition that requires a response when it arrives at a system 자극
Source of the stimulus	This is some entity (a human, a computer system, or any other sensors/actuators) that generated the stimulus 자극의 원천
Artifact	Some artifact is stimulated. This may be a collection of systems, the whole system, or some piece or pieces of it 반응하는 주체
Environment	The stimulus occurs under certain conditions. 자극의 환경.
Response	The response is the output generated or activity undertaken as the result of the arrival of the stimulus 반응
Response Measure	When the response occurs, it should be measurable in some fashion so that the requirement can be tested. 반응 측정.

Architecting software intensive systems-A practitioner's guide(2008)

Sample Performance Scenario

- Users**<Source>** initiate transactions**<Stimulus>** under normal operations**<Environment>**.

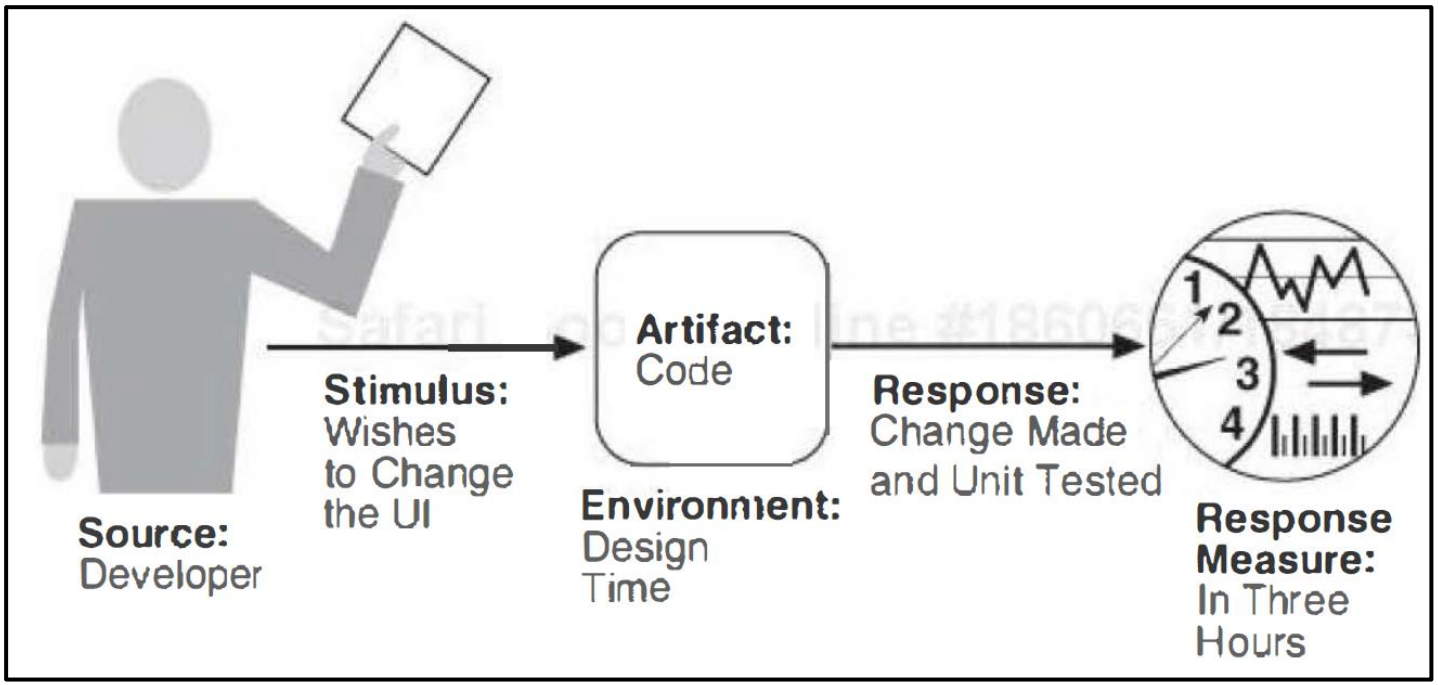
- The system**<Artifact>** processes the transactions**<Response>** with an average latency of two seconds**<Response Measure>**.



Sample Modifiability Scenario

- The developer<**Source**> wishes to change the user interface <**Stimulus**> by modifying the code**<Artifact>** at design time<**Environment**>.

- The modifications are made with no side effects <**Response**> within three hours<**Response Measure**>.

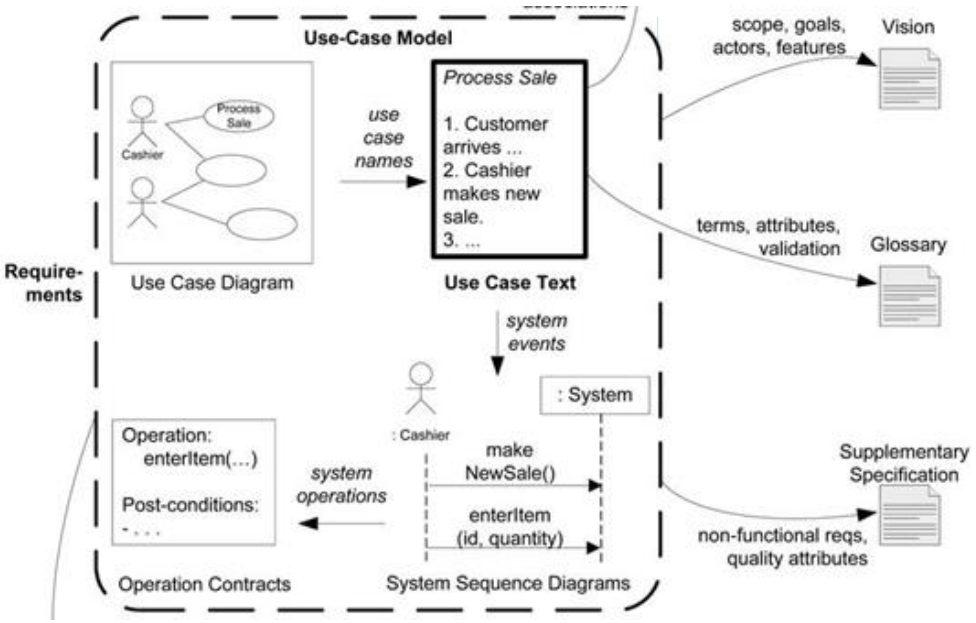


명세서

Software Requirement Specification (SRS)

- a description of a software system to be developed.
- aka. Requirements Analysis Document (RAD)
- Various formats

1. Introduction
   1.1 Purpose of the system
   1.2 Scope of the system
   1.3 Objectives and success criteria of the project
   1.4 Definitions, acronyms, and abbreviations
   1.5 References
   1.6 Overview
2. Current system
3. Proposed system
   3.1 Overview
   3.2 Functional requirements
   3.3 Nonfunctional requirements
      3.3.1 Usability
      3.3.2 Reliability
      ....
      3.3.8 Constraints
   3.4 System models
      3.4.1 Scenarios
      3.4.2 Use case model
      ...



개발될 소프트웨어 시스템의 명세서

Constraint

design 시작전에 이미 결정된 고정된 사전 결정사항.

↳design 시작전에 이미 결정된/사전 고정된 결정사항.

- A constraint is fixed premade decisions that are in place before design begins
  - Business constraints limit decisions about people, process, costs, and schedule.
  - Technical constraints limit decisions about the technology we may use in the software system.

- Each of these exerts forces on the architect and influences the design decisions that the architect makes

- Constraints limit choice, but well-chosen constraints simplify the problem and can make it easier to design a satisficing architecture

Design It! – From programmer to software architect(2017)

Constraint: Examples

Technical Constraints	Business Constraints
Programming Language Choice Anything that runs on the JVM.	**Team Composition and Makeup** Team X will build the XYZ component
Operating System or Platform It must run on Windows, Linux, and BeOS.	**Schedule or Budget** It must be ready in time for the Big Trade Show and cost less than $80,000.
Use of Components or Technology We own DB2 so that's your database.	**Legal Restrictions** There is a 5GB daily limit in our license

Design It! – From programmer to software architect(2017)

Desirable properties of requirements

Unambiguous	It has only one interpretation
Correct	It is one that stakeholder really wants
Complete	It includes all the requirements
Consistent	no subset has conflict
Feasible	It is technically achievable
Traceable	It traces to its source and implementation

Summary

- Software Requirements
  - Larman's FURPS+
  - Functional Requirements (FR)
  - Non-functional Requirements (NFR)
    - Quality Requirements
    - Constraints

- Software Architecture and Requirements
  - Quality Attribute Scenario (QAS)
  - Six parts of QAS
- SRS (Software Requirement Specification)